# 📝 Assignment Walkthrough - Beginner Friendly

## How to Complete Each Assignment

---

## 🔍 Assignment 1: Document Processor (45 min)

### What You're Building

A tool that takes a long document and breaks it into smaller pieces for the AI to understand.

### Why This Matters

AI can't read entire books at once. We need to split documents into "chunks" - small pieces the AI can process.

---

### TODO #1: Clean Text (15 min)

**Location:** Line ~40 in `document_processor.py`

**What it does:** Removes messy formatting from text

**Example:**

```
Input:  "Hello    World!!!\n\n\tHow  are   you?"
Output: "Hello World!!! How are you?"
```

**Your Task:** Fill in this function:

```
def clean_text(self, text: str) -> str:
    # Step 1: Replace multiple spaces with single space
```

```
    cleaned = re.sub(r'\s+', ' ', text)

    # Step 2: Remove weird characters (keep only letters, numbers, punctuat
    cleaned = re.sub(r'[^\w\s.,!?;:\-\'\"()]', '', cleaned)

    # Step 3: Remove spaces at start/end
    return cleaned.strip()
```

**Understanding the code:**

- `re.sub(r'\s+', ' ', text)` means: "Find any whitespace ( `\s+` ) and replace with single space ( )"
- `r'[^\w\s.,!?;:\-\'\"()]'` means: "Find anything that's NOT a word, space, or basic punctuation"
- `.strip()` removes spaces from start and end

**Test it:**

```
python document_processor.py
```

**Expected output:**

```
Testing text cleaning...
Original length: 345, Cleaned length: 320
✓ Text cleaned
```

---

## TODO #2: Chunk Document (20 min)

**Location:** Line ~60 in `document_processor.py`

**What it does:** Splits long text into overlapping pieces

**Example:**

```
Text: "The cat sat on the mat. The dog ran in the park."

Chunk 1: "The cat sat on the mat."
Chunk 2: "on the mat. The dog ran"  ← Notice overlap!
Chunk 3: "The dog ran in the park."
```

**Why overlap?** So context isn't lost between chunks!

**Your Task:**

```
def chunk_document(self, text: str, chunk_size: int = CHUNK_SIZE,
                   overlap: int = CHUNK_OVERLAP) -> List[Dict]:
    chunks = []
    start = 0
    chunk_id = 0

    # Keep going until we've processed all text
    while start < len(text):
        # Extract a chunk
        end = start + chunk_size
        chunk_text = text[start:end]

        # Save the chunk with metadata
        chunks.append({
            'text': chunk_text,
            'chunk_id': chunk_id,
            'position': start,
            'length': len(chunk_text)
        })

        # Move to next chunk (with overlap)
        start = end - overlap
        chunk_id += 1

        # Stop if we're at the end
        if end >= len(text):
            break

    return chunks
```

**Understanding the code:**

- `start = 0` - Begin at the start of the text
- `end = start + chunk_size` - Go 500 characters forward (default chunk size)
- `start = end - overlap` - Move back 50 characters for overlap
- Loop continues until we've processed all text

**Test it:**

```
python document_processor.py
```

**Expected output:**

```
Testing chunking...
Created 3 chunks
Chunk 0: Artificial Intelligence has revolutionized many...
Chunk 1: ...fields. Machine learning, a subset of AI...
```

---

## TODO #3: Create Embeddings (10 min)

**Location:** Line ~90 in `document_processor.py`

**What it does:** Converts text into numbers (vectors) that AI can understand

**Example:**

```
Text: "Hello world"
Embedding: [0.234, -0.156, 0.891, ... ] (384 numbers!)
```

**Why?** AI works with numbers, not words. Similar words get similar numbers.

**Your Task:**

```python
def create_embeddings(self, chunks: List[Dict]) -> List[List[float]]:
    # Step 1: Get just the text from each chunk
    texts = [chunk['text'] for chunk in chunks]

    # Step 2: Convert to embeddings using the AI model
    embeddings = self.embedding_model.encode(texts)

    # Step 3: Convert to list format (ChromaDB needs lists, not numpy array
    return embeddings.tolist()
```

**Understanding the code:**

- `[chunk['text'] for chunk in chunks]` - This is a "list comprehension" - it extracts text from each chunk
- `self.embedding_model.encode(texts)` - The AI model converts text to numbers
- `.tolist()` - Converts numpy array to Python list

**Test it:**

```
python document_processor.py
```

**Expected output:**

```
Testing full ingestion...
Processing document: sample_doc
✓ Text cleaned (345 characters)
✓ Created 3 chunks
✓ Generated 3 embeddings
✓ Stored in ChromaDB

Success! Ingested 3 chunks
```

✅ **Assignment 1 Complete!**

---

# 🎯 Assignment 2: Relevance Scorer (45 min)

## What You're Building

A system that finds the most relevant chunks for a user's question.

## Why This Matters

If someone asks "What is AI?", we need to find the most relevant parts of the document, not random chunks.

---

## TODO #1: Cosine Similarity (15 min)

**Location:** Line ~35 in `relevance_scorer.py`

**What it does:** Measures how similar two embeddings are

**Example:**

```
Question embedding: [0.5, 0.3, 0.2]
Chunk embedding:    [0.6, 0.4, 0.1]
```

```
Similarity: 0.95 (very similar!)
```

**Your Task:**

```python
def cosine_similarity(self, vec1: List[float], vec2: List[float]) -> float:
    # Convert lists to numpy arrays (easier math)
    a = np.array(vec1)
    b = np.array(vec2)

    # Calculate dot product (multiply corresponding elements and sum)
    dot_product = np.dot(a, b)

    # Calculate magnitudes (length of vectors)
    norm_a = np.linalg.norm(a)
    norm_b = np.linalg.norm(b)

    # Cosine similarity = dot product / (norm_a * norm_b)
    if norm_a == 0 or norm_b == 0:
        return 0.0

    return dot_product / (norm_a * norm_b)
```

**Understanding the math:**

- **Dot product**: Sum of (a[0]×b[0] + a[1]×b[1] + a[2]×b[2] + ...)
- **Norm**: Length of vector √(a[0]² + a[1]² + a[2]² + ...)
- **Result**: Number between -1 and 1 (we use 0 to 1)
    - 1.0 = identical vectors
    - 0.5 = somewhat similar
    - 0.0 = completely different

**Test it:**

```
python relevance_scorer.py
```

**Expected output:**

```
Testing cosine similarity...
Same vectors: 1.0 (should be ~1.0)
Orthogonal vectors: 0.0 (should be ~0.0)
```

---

## TODO #2: Keyword Overlap (10 min)

**Location:** Line ~60 in `relevance_scorer.py`

**What it does:** Counts how many words match between question and document

**Example:**

```
Query: "machine learning algorithms"
Doc:   "Machine learning uses various algorithms"
Overlap: 2 words match (machine, learning)
Score: 0.4 (2 ÷ 5 total unique words)
```

**Your Task:**

```python
def keyword_overlap_score(self, query: str, document: str) -> float:
    # Step 1: Convert to lowercase and split into words
    query_words = set(query.lower().split())
    doc_words = set(document.lower().split())

    # Step 2: Find matching words
    intersection = query_words & doc_words  # Words in BOTH
    union = query_words | doc_words          # Words in EITHER

    # Step 3: Calculate Jaccard similarity
    if len(union) == 0:
        return 0.0

    return len(intersection) / len(union)
```

**Understanding the code:**

- `set()` removes duplicates - “the the the” becomes just “the”
- `&` finds intersection (words in both)
- `|` finds union (all unique words)
- **Jaccard similarity** = size of intersection ÷ size of union

**Test it:**

```
python relevance_scorer.py
```

**Expected output:**

```
Testing keyword overlap...
Relevant doc overlap: 0.375
Irrelevant doc overlap: 0.0
```

---

## TODO #3 & #4: Calculate and Filter Scores (20 min)

**Location:** Lines ~85 and ~115 in `relevance_scorer.py`

**What it does:** Combines multiple scores and filters bad results

**Your Task for calculate_relevance_scores:**

```
def calculate_relevance_scores(self, query: str, results: Dict) -> List[Dic
    scored_results = []

    # Check if we have results
    if not results['documents'] or len(results['documents'][0]) == 0:
        return scored_results

    # Loop through each result
    for i, doc in enumerate(results['documents'][0]):
        # ChromaDB gives us distance; convert to similarity
        cosine_sim = 1 - results['distances'][0][i]

        # Calculate keyword overlap
        keyword_score = self.keyword_overlap_score(query, doc)

        # Combine scores (70% semantic, 30% keyword)
        combined = 0.7 * cosine_sim + 0.3 * keyword_score

        # Create result dictionary
        scored_results.append({
            'text': doc,
            'cosine_similarity': cosine_sim,
            'keyword_overlap': keyword_score,
            'combined_score': combined,
            'source': results['metadatas'][0][i].get('source', 'unknown'),
            'metadata': results['metadatas'][0][i]
        })

    return scored_results
```

**Your Task for filter_by_threshold:**

```
def filter_by_threshold(self, results: List[Dict],
                        threshold: float = SIMILARITY_THRESHOLD,
                        score_key: str = 'combined_score') -> List[Dict]:
    # Filter results above threshold
    filtered = [r for r in results if r.get(score_key, 0) >= threshold]

    # Sort by score (highest first)
    filtered = sorted(filtered, key=lambda x: x[score_key], reverse=True)

    return filtered
```

**Understanding the code:**

- We weight semantic similarity higher (0.7) because it's usually more accurate
- List comprehension `[r for r in results if ...]` filters the list
- `sorted(..., reverse=True)` sorts highest to lowest

✅ **Assignment 2 Complete!**

---

# 🛡️ Assignment 3: Guardrails (60 min)

## What You're Building

A safety system that detects private information and toxic content.

## Why This Matters

We don't want the AI to accidentally share someone's email address or respond to toxic queries!

---

## TODO #1: Detect PII (20 min)

**Location:** Line ~40 in `guardrails.py`

**What it does:** Finds emails, phone numbers, SSNs, credit cards

**Example:**

```
Text: "Email me at john@example.com or call 555-1234"
Found: {'email': ['john@example.com'], 'phone': ['555-1234']}
```

**Your Task:**

```python
def detect_pii(self, text: str) -> Dict[str, List[str]]:
    found_pii = {}

    # Loop through each PII pattern from config
    for pii_type, pattern in PII_PATTERNS.items():
        # Use regex to find all matches
        matches = re.findall(pattern, text)

        # If we found any, add to results
        if matches:
            found_pii[pii_type] = matches

    return found_pii
```

**Understanding the code:**

- `PII_PATTERNS` comes from `config.py` - it has regex patterns for emails, phones, etc.
- `re.findall(pattern, text)` finds all matches in the text
- We only add to `found_pii` if we actually found something

**Test it:**

```
python guardrails.py
```

---

## TODO #2: Check Toxicity (20 min)

**Location:** Line ~70 in `guardrails.py`

**What it does:** Uses AI to detect toxic/offensive content

**Your Task:**

```
def check_toxicity(self, text: str) -> Tuple[bool, float, Dict]:
    if not self.toxicity_enabled:
        return False, 0.0, {"note": "Toxicity detection disabled"}

    if not text or len(text.strip()) == 0:
        return False, 0.0, {"note": "Empty text"}

    try:
        # Limit to 512 characters (model limit)
        text = text[:512]

        # Get toxicity scores from model
        results = self.toxicity_detector(text)

        # Find the toxic score
        toxic_score = 0.0
        for result in results[0]:
            if result['label'] == 'toxic':
                toxic_score = result['score']
                break

        # Check if above threshold
        is_toxic = toxic_score > TOXICITY_THRESHOLD

        return is_toxic, toxic_score, {'labels': results[0]}

    except Exception as e:
        return False, 0.0, {"error": str(e)}
```

---

### TODO #3 & #4: Validation and Sanitization (20 min)

Complete the `validate_query` and `sanitize_text` functions using the patterns shown above.

**✅ Assignment 3 Complete!**

---

## 🎨 Assignment 4: Streamlit App (60 min)

---

## What You're Building

A web interface where users can upload documents and ask questions.

## Your Tasks:

1. Process uploaded files
2. Validate queries with guardrails
3. Search for relevant chunks
4. Generate AI responses
5. Display results

**Follow the TODOs in `rag_app.py` and use the hints provided!**

**✅ Assignment 4 Complete - You're Done!**

---

# 🎉 Celebrate!

You just built a complete RAG system! That's impressive for a beginner.

**What you learned:**

- Python programming
- Working with AI models
- Vector databases
- Web development
- Text processing
- Software safety

These are real skills used in the AI industry!

**Next steps:**

- Try the bonus challenges
- Show your friends
- Add it to your portfolio
- Keep learning!